NAME : N ASHWITHA REDDY

Roll number:2303A52089

BATCH -40

AI-ASSISTED CODING ASSIGNMENT 8.1

```python
# Task Description #1 (Password Strength Validator - Apply AI in
# Security Context)
# • Task: Apply AI to generate at least 3 assert test cases for
# is_strong_password(password) and implement the validator
# function.
# • Requirements:
# o Password must have at least 8 characters.
# o Must include uppercase, lowercase, digit, and special
# character.
# o Must not contain spaces.
# Example Assert Test Cases:
# assert is_strong_password("Abcd@123") == True
# assert is_strong_password("abcd123") == False
# assert is_strong_password("ABCD@1234") == True
# Expected Output #1:
# • Password validation logic passing all AI-generated test cases code
# PROMPT: Generate assert test cases for a Python function is_strong_password(password) that checks if a password is strong based on criteria: at least 8 characters,
#  includes uppercase, lowercase, digit, special character, and no spaces. Include edge cases like missing character types and spaces.
import re
def is_strong_password(password):
    if len(password) < 8:
        return False
    if not re.search(r'[A-Z]', password):
        return False
    if not re.search(r'[a-z]', password):
        return False
    if not re.search(r'[0-9]', password):
        return False
    if not re.search(r'[@$!%*?&]', password):
        return False
    if ' ' in password:
        return False
    return True
# AI-generated assert test cases
assert is_strong_password("Abcd@123") == True
assert is_strong_password("abcd123") == False
assert is_strong_password("ABCD@1234a") == True
assert is_strong_password("Abcdefg1") == False  # Missing special character
assert is_strong_password("Abc@ 123") == False  # Contains space
assert is_strong_password("Abc@123") == False  # Less than 8 characters
assert is_strong_password("Abcdefg@") == False  # Missing digit
assert is_strong_password("12345678") == False  # Missing uppercase, lowercase, and special character
assert is_strong_password("ABCDEFGH") == False  # Missing lowercase, digit, and special character
assert is_strong_password("abcdefgh") == False  # Missing uppercase, digit, and special character
print("All test cases passed!")
```

```python
# Task Description #2 (Number Classification with Loops – Apply
# AI for Edge Case Handling)
# • Task: Use AI to generate at least 3 assert test cases for a
# classify_number(n) function. Implement using loops
# Requirements:
# o Classify numbers as Positive, Negative, or Zero.
# o Handle invalid inputs like strings and None.
# o Include boundary conditions (-1, 0, 1).
# Example Assert Test Cases:
# assert classify_number(10) == "Positive"
# assert classify_number(-5) == "Negative"
# assert classify_number(0) == "Zero"
# Expected Output #2:
# • Classification logic passing all assert tests.
# PROMPT: Generate assert test cases for a Python function classify_number(n) that classifies numbers as Positive, Negative, or Zero.
#  Handle invalid inputs like strings and None, and include boundary conditions (-1, 0, 1).
def classify_number(n):
    try:
        # Handle None explicitly
        if n is None:
            return "Invalid input"
        # Convert to string to validate using loop
        n_str = str(n).strip()
        # Check valid numeric format using loop
        decimal_count = 0
        for i, ch in enumerate(n_str):
            if ch == '.':
                decimal_count += 1
                if decimal_count > 1:
                    return "Invalid input"
            elif ch == '-' and i == 0:
                continue
            elif not ch.isdigit():
                return "Invalid input"
        n = float(n_str)
        if n > 0:
            return "Positive"
        elif n < 0:
            return "Negative"
        else:
            return "Zero"
    except (ValueError, TypeError):
        return "Invalid input"
assert classify_number(10) == "Positive"
assert classify_number(-5) == "Negative"
assert classify_number(0) == "Zero"
assert classify_number("abc") == "Invalid input"
assert classify_number(None) == "Invalid input"
assert classify_number(-1) == "Negative"
assert classify_number(1) == "Positive"
assert classify_number("12.5") == "Positive"
assert classify_number("-0.5") == "Negative"
assert classify_number("12.3.4") == "Invalid input"
print("All test cases passed!")
```

```python
# Task Description #3 (Anagram Checker - Apply AI for String
# Analysis)
# • Task: Use AI to generate at least 3 assert test cases for
# is_anagram(str1, str2) and implement the function.
# • Requirements:
# o Ignore case, spaces, and punctuation.
# o Handle edge cases (empty strings, identical words).
# Example Assert Test Cases:
# assert is_anagram("listen", "silent") == True
# assert is_anagram("hello", "world") == False
# assert is_anagram("Dormitory", "Dirty Room") == True
# Expected Output #3:
# • Function correctly identifying anagrams and passing all AI-
# generated tests.
# Generate Python assert-based test cases for an anagram checker function is_anagram(str1, str2). Ignore case, spaces, and punctuation.
#  Include edge cases like empty strings and identical words.
import string
def is_anagram(str1, str2):
    # Remove spaces and punctuation, convert to lowercase
    clean1 = ''.join(ch.lower() for ch in str1 if ch.isalnum())
    clean2 = ''.join(ch.lower() for ch in str2 if ch.isalnum())
    return sorted(clean1) == sorted(clean2)
# AI-generated assert test cases
assert is_anagram("listen", "silent") == True
assert is_anagram("hello", "world") == False
assert is_anagram("Dormitory", "Dirty Room") == True
assert is_anagram("The Eyes", "They See") == True
assert is_anagram("", "") == True   # Empty strings
assert is_anagram("a", "a") == True
assert is_anagram("rat", "car") == False
print("All anagram test cases passed!")
```

```python
# Task Description #4 (Inventory Class – Apply AI to Simulate Real-
# World Inventory System)
# • Task: Ask AI to generate at least 3 assert-based tests for an
# Inventory class with stock management.
# • Methods:
# o add_item(name, quantity)
# o remove_item(name, quantity)
# o get_stock(name)
# Example Assert Test Cases:
# inv = Inventory()
# inv.add_item("Pen", 10)
# assert inv.get_stock("Pen") == 10
# inv.remove_item("Pen", 5)
# assert inv.get_stock("Pen") == 5
# inv.add_item("Book", 3)
# assert inv.get_stock("Book") == 3
# Expected Output #4:
# • Fully functional class passing all assertions.
# PROMPT: Generate assert-based test cases for a Python Inventory class with methods add_item(name, quantity), remove_item(name, quantity),
#  and get_stock(name). Include edge cases like removing more than available stock and checking non-existing items.
class Inventory:
    def __init__(self):
        self.stock = {}
    def add_item(self, name, quantity):
        if quantity <= 0:
            return
        self.stock[name] = self.stock.get(name, 0) + quantity
    def remove_item(self, name, quantity):
        if name in self.stock and quantity > 0:
            if self.stock[name] >= quantity:
                self.stock[name] -= quantity
    def get_stock(self, name):
        return self.stock.get(name, 0)
# AI-generated assert test cases
inv = Inventory()
inv.add_item("Pen", 10)
assert inv.get_stock("Pen") == 10
inv.remove_item("Pen", 5)
assert inv.get_stock("Pen") == 5
inv.add_item("Book", 3)
assert inv.get_stock("Book") == 3
inv.remove_item("Pen", 10)  # Removing more than available
assert inv.get_stock("Pen") == 5  # Should remain unchanged
assert inv.get_stock("Pencil") == 0  # Non-existing item
print("All inventory test cases passed!")
```

```python
# Task Description #5 (Date Validation & Formatting - Apply AI for
# Data Validation)
# • Task: Use AI to generate at least 3 assert test cases for
# validate_and_format_date(date_str) to check and convert
# dates.
# • Requirements:
# o Validate "MM/DD/YYYY" format.
# o Handle invalid dates.
# o Convert valid dates to "YYYY-MM-DD".
# Example Assert Test Cases:
# assert validate_and_format_date("10/15/2023") == "2023-10-15"
# assert validate_and_format_date("02/30/2023") == "Invalid Date"
# assert validate_and_format_date("01/01/2024") == "2024-01-01"
# Expected Output #5:
# • Function passes all AI-generated assertions and handles edge
# cases.
# PROMPT:Generate assert test cases for a Python function validate_and_format_date(date_str) that validates "MM/DD/YYYY" format,
#  handles invalid dates, and converts valid dates to "YYYY-MM-DD". Include edge cases like invalid month, invalid day, and leap year.
from datetime import datetime
def validate_and_format_date(date_str):
    try:
        date_obj = datetime.strptime(date_str, "%m/%d/%Y")
        return date_obj.strftime("%Y-%m-%d")
    except ValueError:
        return "Invalid Date"
# AI-generated assert test cases
assert validate_and_format_date("10/15/2023") == "2023-10-15"
assert validate_and_format_date("02/30/2023") == "Invalid Date"
assert validate_and_format_date("01/01/2024") == "2024-01-01"
assert validate_and_format_date("13/01/2023") == "Invalid Date"
assert validate_and_format_date("02/29/2024") == "2024-02-29"  # Leap year
assert validate_and_format_date("02/29/2023") == "Invalid Date"
print("All date validation test cases passed!")
```

```
PS C:\Users\hp\Desktop\AIAC> python A81.py
All test cases passed!
All test cases passed!
All anagram test cases passed!
All inventory test cases passed!
All date validation test cases passed!
PS C:\Users\hp\Desktop\AIAC>
```

A81.py > ...

```python
# An online shopping company wants to build an order tracking and sales analysis system to maintain customer purchase records and analyze product sales.You are required to develop a python program using classes,constructors, loops, and conditional statements  and file handling.Each order should store order ID ,customer name,product name,quantity,price per item,and order date,whenever a new order is placed the details must be saved permanantly in the file. The program should allow the operator to repeatedly perform operations such as adding a new order ,viewing all orders, searching an order by order id,and updating the quantity and price.The system must automatically calculate the total amount for each order and generate a sales analyis report showing the total revenue ,most sold product,and the highest value order.The program should continue running until the user chooses to exit.


# PROMPT:Generate a Python program for an online shopping order tracking and sales analysis system using classes, constructors, loops, conditional statements, and file handling.
# Each order should store:
# Order ID
# Customer Name
# Product Name
# Quantity
# Price per item
# Order Date
# Requirements:
# Save orders permanently in a file.
# Allow repeated operations:
# Add new order
# View all orders
# Search order by ID
# Update quantity and price
# Automatically calculate total amount
# Generate sales report including:
# Total revenue
# Most sold product
# Highest value order
# Program should run until user chooses to exit.

import os
class Order:
    def __init__(self, order_id, customer_name, product_name, quantity, price, order_date):
        self.order_id = order_id
        self.customer_name = customer_name
        self.product_name = product_name
        self.quantity = int(quantity)
        self.price = float(price)
        self.order_date = order_date
    def total_amount(self):
        return self.quantity * self.price
    def to_string(self):
        return f"{self.order_id},{self.customer_name},{self.product_name},{self.quantity},{self.price},{self.order_date}\n"
class OrderSystem:
    def __init__(self, filename="orders.txt"):
        self.filename = filename
        if not os.path.exists(self.filename):
            open(self.filename, "w").close()
```

Ln 258, Col 20 Spaces: 4 UTF-8 CRLF {} Python

```python
class OrderSystem:
    def add_order(self):
        order_id = input("Enter Order ID: ")
        customer = input("Enter Customer Name: ")
        product = input("Enter Product Name: ")
        quantity = input("Enter Quantity: ")
        price = input("Enter Price per Item: ")
        date = input("Enter Order Date (DD/MM/YYYY): ")
        order = Order(order_id, customer, product, quantity, price, date)
        with open(self.filename, "a") as file:
            file.write(order.to_string())
        print("Order added successfully!")
    def view_orders(self):
        with open(self.filename, "r") as file:
            lines = file.readlines()
            if not lines:
                print("No orders found.")
                return
            for line in lines:
                data = line.strip().split(",")
                order = Order(*data)
                print(f"ID: {order.order_id}, Customer: {order.customer_name}, "
                      f"Product: {order.product_name}, Quantity: {order.quantity}, "
                      f"Price: {order.price}, Date: {order.order_date}, "
                      f"Total: {order.total_amount()}")
    def search_order(self):
        search_id = input("Enter Order ID to search: ")
        found = False
        with open(self.filename, "r") as file:
            for line in file:
                data = line.strip().split(",")
                if data[0] == search_id:
                    order = Order(*data)
                    print("Order Found:")
                    print(f"Customer: {order.customer_name}")
                    print(f"Product: {order.product_name}")
                    print(f"Total Amount: {order.total_amount()}")
                    found = True
                    break
        if not found:
            print("Order not found.")
    def update_order(self):
        update_id = input("Enter Order ID to update: ")
        updated_lines = []
        found = False
        with open(self.filename, "r") as file:
            lines = file.readlines()
```

```python
class OrderSystem:
    def update_order(self):
        for line in lines:
            data = line.strip().split(",")
            if data[0] == update_id:
                quantity = input("Enter new Quantity: ")
                price = input("Enter new Price per Item: ")
                data[3] = quantity
                data[4] = price
                found = True
                print("Order updated successfully!")
            updated_lines.append(",".join(data) + "\n")
        with open(self.filename, "w") as file:
            file.writelines(updated_lines)
        if not found:
            print("Order ID not found.")
    def sales_report(self):
        total_revenue = 0
        product_sales = {}
        highest_order = 0
        highest_order_id = None
        with open(self.filename, "r") as file:
            for line in file:
                data = line.strip().split(",")
                order = Order(*data)
                total = order.total_amount()
                total_revenue += total
                product_sales[order.product_name] = product_sales.get(order.product_name, 0) + order.quantity
                if total > highest_order:
                    highest_order = total
                    highest_order_id = order.order_id
        print("\n--- Sales Analysis Report ---")
        print(f"Total Revenue: {total_revenue}")
        if product_sales:
            most_sold = max(product_sales, key=product_sales.get)
            print(f"Most Sold Product: {most_sold}")
        else:
            print("Most Sold Product: None")
        print(f"Highest Value Order ID: {highest_order_id}")
        print("----------------------------")
# Main Program Loop
system = OrderSystem()
while True:
    print("\n1. Add Order")
```

A81.py > ...

```python
    while True:
        print("\n1. Add Order")
        print("2. View Orders")
        print("3. Search Order")
        print("4. Update Order")
        print("5. Sales Report")
        print("6. Exit")
        choice = input("Enter your choice: ")
        if choice == "1":
            system.add_order()
        elif choice == "2":
            system.view_orders()
        elif choice == "3":
            system.search_order()
        elif choice == "4":
            system.update_order()
        elif choice == "5":
            system.sales_report()
        elif choice == "6":
            print("Exiting program...")
            break
        else:
            print("Invalid choice. Try again.")

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORT

```
1. Add Order
2. View Orders
3. Search Order
4. Update Order
5. Sales Report
6. Exit
Enter your choice: 1
Enter Order ID: 123
Enter Customer Name: humera
Enter Product Name: eggs
Enter Quantity: 4
Enter Price per Item: 2
Enter Order Date (DD/MM/YYYY): 11 02 2026
Order added successfully!
```

```
1. Add Order
2. View Orders
3. Search Order
4. Update Order
5. Sales Report
6. Exit
Enter your choice: 2
ID: 123, Customer: humera, Product: eggs, Quantity: 4, Price: 2.0, Date: 11 02 2026, Total: 8.0

1. Add Order
2. View Orders
3. Search Order
4. Update Order
5. Sales Report
6. Exit
Enter your choice: 3
Enter Order ID to search: 123
Order Found:
Customer: humera
Product: eggs
Total Amount: 8.0

1. Add Order
2. View Orders
3. Search Order
4. Update Order
5. Sales Report
6. Exit
Enter your choice: 4
Enter Order ID to update: 123
Enter new Quantity: 5
Enter new Price per Item: 2
Order updated successfully!

1. Add Order
2. View Orders
3. Search Order
4. Update Order
5. Sales Report
6. Exit
```

```
6. Exit
Enter your choice: 4
Enter Order ID to update: 123
Enter new Quantity: 5
Enter new Price per Item: 2
Order updated successfully!

1. Add Order
2. View Orders
3. Search Order
4. Update Order
5. Sales Report
6. Exit
Enter your choice: 5

--- Sales Analysis Report ---
Total Revenue: 10.0
Most Sold Product: eggs
Highest Value Order ID: 123
----------------------------

1. Add Order
2. View Orders
3. Search Order
4. Update Order
5. Sales Report
6. Exit
Enter your choice: 6
Exiting program...
PS C:\Users\hp\Desktop\AIAC>
```

ANALYSIS:

#Task 1 – Password Strength Validator

This program checks whether a password meets security requirements like minimum length, uppercase, lowercase, digit, and special character. Regular expressions are used for validation. Edge cases like missing characters and spaces are handled. The function passes all assert test cases.

#Task 2 – Number Classification

The function classifies a number as Positive, Negative, or Zero using conditional statements. Exception handling is used to manage invalid inputs like strings and None. Boundary cases (-1, 0, 1) are tested. All assertions are successfully passed.

#Task 3 – Anagram Checker

This program checks if two strings are anagrams by ignoring case, spaces, and punctuation. It cleans the strings and compares sorted characters. Edge cases like empty strings and identical words are handled. The function passes all test cases.

#Task 4 – Inventory Class

The Inventory class manages stock using object-oriented programming. It supports adding, removing, and checking stock using methods. Conditional logic prevents negative stock. All assert-based tests are successfully validated.

#Task 5 – Date Validation & Formatting

The function validates dates in "MM/DD/YYYY" format using the datetime module. It converts valid dates to "YYYY-MM-DD" and handles invalid dates using exception handling. Leap year cases are also considered. All test cases pass correctly

#Task 6 – Order Tracking & Sales Analysis System

This program uses classes and file handling to store and manage order records permanently. It supports adding, searching, updating orders, and generating sales reports. The system calculates total revenue, most sold product, and highest value order. The program runs continuously until the user exits.